Semana 1 · Base del laboratorio y OpenSSL 3.5

Fase de la propuesta: revisión y validación con el cliente (semanas 1-2).

Objetivo	Dejar la Raspberry lista para la propuesta aprobada: GitHub seguro, repositorio reorganizado, código nuevo instalado y verificación de que OpenSSL 3.5 hace el handshake híbrido.
Dedicación estimada	4–6 horas.
Antes de empezar	SSH a la Raspberry (usuario usuariok). El archivo codigo_integradora.zip en su laptop.
Al terminar tendrán	Repositorio con la estructura nueva, certificado del servidor y la evidencia `docs/evidencias/entorno_*.txt`.

> **Lo que ya tienen hecho (no lo repitan)**
>
> Sistema operativo, SSH, entorno virtual en `~/proyecto-kyber-rpi/src/venv`, librerías del INA219, liboqs-python y la conexión del INA219. Esta semana se **reorganiza** eso hacia la propuesta nueva.

Paso 1. Asegurar GitHub con una clave SSH

Si todavía no revocaron el token que apareció en el PDF, háganlo primero (GitHub → Settings → Developer settings → Personal access tokens → Delete). Luego, en la Raspberry:

```
ssh-keygen -t ed25519 -C "integradoraPi4"      # Enter a todo
cat ~/.ssh/id_ed25519.pub                      # copien esta línea completa
```

En GitHub: **Settings → SSH and GPG keys → New SSH key**, peguen la línea y guarden. Después:

```
ssh -T git@github.com                          # debe decir: Hi kali2010-x! ...
cd ~/proyecto-kyber-rpi
git remote set-url origin git@github.com:kali2010-x/proyecto-kyber-rpi.git
git log -p | grep -c "ghp_"                    # debe dar 0 (ningún token en el historial)
```

Paso 2. Actualizar el sistema e instalar herramientas

```
sudo apt update && sudo apt full-upgrade -y
sudo apt install -y git tmux i2c-tools python3-dev build-essential \
     autoconf automake libtool stress-ng unzip zip
sudo reboot
```

Paso 3. Mover lo anterior a legacy/ e instalar el código nuevo

Primero se mueve lo de la propuesta anterior (sin borrar nada) y luego se descomprime el paquete. El entorno virtual `src/venv` se queda donde está.

```
# en la LAPTOP (PowerShell o terminal), desde la carpeta donde está el zip:
scp codigo_integradora.zip usuariok@192.168.0.106:~

# en la RASPBERRY:
cd ~/proyecto-kyber-rpi
mkdir -p legacy/src legacy/data legacy/docs
git mv src/*.py src/requirements.txt legacy/src/
git mv data/*.csv legacy/data/
git mv docs/*.md docs/*.txt legacy/docs/
unzip -o ~/codigo_integradora.zip -d ~/proyecto-kyber-rpi
ls                                     # deben ver: pi esp32 sim herramientas certs data docs legacy
```

> **Si falla**
>
> Si `git mv` dice que un archivo no existe, sáltenlo: significa que ya no estaba. Revisen con `git status` que nada se haya borrado.

Paso 4. Instalar las dependencias de Python en el venv

```
source ~/proyecto-kyber-rpi/src/venv/bin/activate
pip install -r ~/proyecto-kyber-rpi/requirements.txt
python3 -c "import smbus2, gpiod, pandas, scipy, matplotlib; print('dependencias OK')"
```

> **Si falla**
>
> Si `gpiod` no compila: `sudo apt install -y python3-dev build-essential` y repitan. Si scipy o pandas tardan mucho, es normal la primera vez (piwheels).

Paso 5. Crear el certificado del servidor TLS

El servidor (Raspberry) se autentica con un certificado ECDSA P-256 autofirmado. El ESP32 llevará una copia del certificado público para verificarlo. La clave privada nunca se sube (está en .gitignore).

```
cd ~/proyecto-kyber-rpi
openssl req -x509 -newkey ec -pkeyopt ec_paramgen_curve:P-256 -nodes \
    -keyout certs/servidor.key -out certs/servidor.pem -days 365 -subj "/CN=servidor-borde"
openssl x509 -in certs/servidor.pem -noout -subject -enddate
git status --short certs/            # debe aparecer servidor.pem pero NO servidor.key
```

Paso 6. Verificar OpenSSL 3.5 y el handshake híbrido

```
cd ~/proyecto-kyber-rpi && source src/venv/bin/activate
bash herramientas/verificar_entorno.sh
```

Revisen en la salida (queda guardada en `docs/evidencias/entorno_<fecha>.txt`):

- OpenSSL **3.5.x o superior** y la lista de **ML-KEM-512/768/1024**.
- En la prueba con X25519MLKEM768: `Negotiated TLS1.3 group: X25519MLKEM768`.
- En la prueba con X25519: `Peer Temp Key: X25519, 253 bits`.
- En Python: ClientHello **mayor a 1.200 B** con el grupo híbrido y **menor a 600 B** con el clásico.
- `i2cdetect -y 1` muestra el INA219 en **40**.

> **Si OpenSSL es menor a 3.5**
>
> Detengan aquí y avísenme con la salida de `openssl version` y `cat /etc/os-release`. Sin OpenSSL 3.5 la Raspberry no puede hacer el intercambio híbrido.

Paso 7. Bitácora, decisiones y commit

```
cd ~/proyecto-kyber-rpi
echo "# Bitácora del proyecto" > docs/bitacora.md
nano docs/bitacora.md                # anoten fecha, qué hicieron y qué problemas hubo
git add -A
git commit -m "Semana 1: estructura nueva, certificado y verificación de OpenSSL"
git push
```

Criterio de cierre de la semana

- ☐ El token de GitHub está revocado y el push funciona por SSH.
- ☐ Los scripts anteriores están en legacy/ y el código nuevo en pi/, esp32/, sim/, herramientas/.
- ☐ pip instaló las dependencias sin errores.
- ☐ certs/servidor.pem existe y servidor.key NO está en Git.
- ☐ verificar_entorno.sh muestra OpenSSL ≥ 3.5, X25519MLKEM768 negociado y el INA219 en 0x40.
- ☐ Commit y push hechos; bitácora creada.